

Cloud BigTable 簡介

來源：<https://codelabs.developers.google.com/codelabs/cloud-bigtable-intro-java?hl=zh-tw>

步驟數：11

在這個程式碼研究室中，您將會瞭解如何以 Java HBase 用戶端接觸 Cloud Bigtable。您需要載入資料，然後執行查詢，並在地圖上繪製資料。

目錄

1. [1. 簡介](#)
2. [2. 關於資料集](#)
3. [3. 結構定義設計](#)
4. [4. 建立執行個體、資料表和系列](#)
5. [5. 匯入資料](#)
6. [6. 取得程式碼](#)
7. [7. 執行查詢](#)
8. [8. 執行掃描](#)
9. [9. 介紹篩選器](#)
10. [10. 執行多範圍掃描](#)
11. [11. 完成設定](#)

步驟 1 · 約 0 分鐘

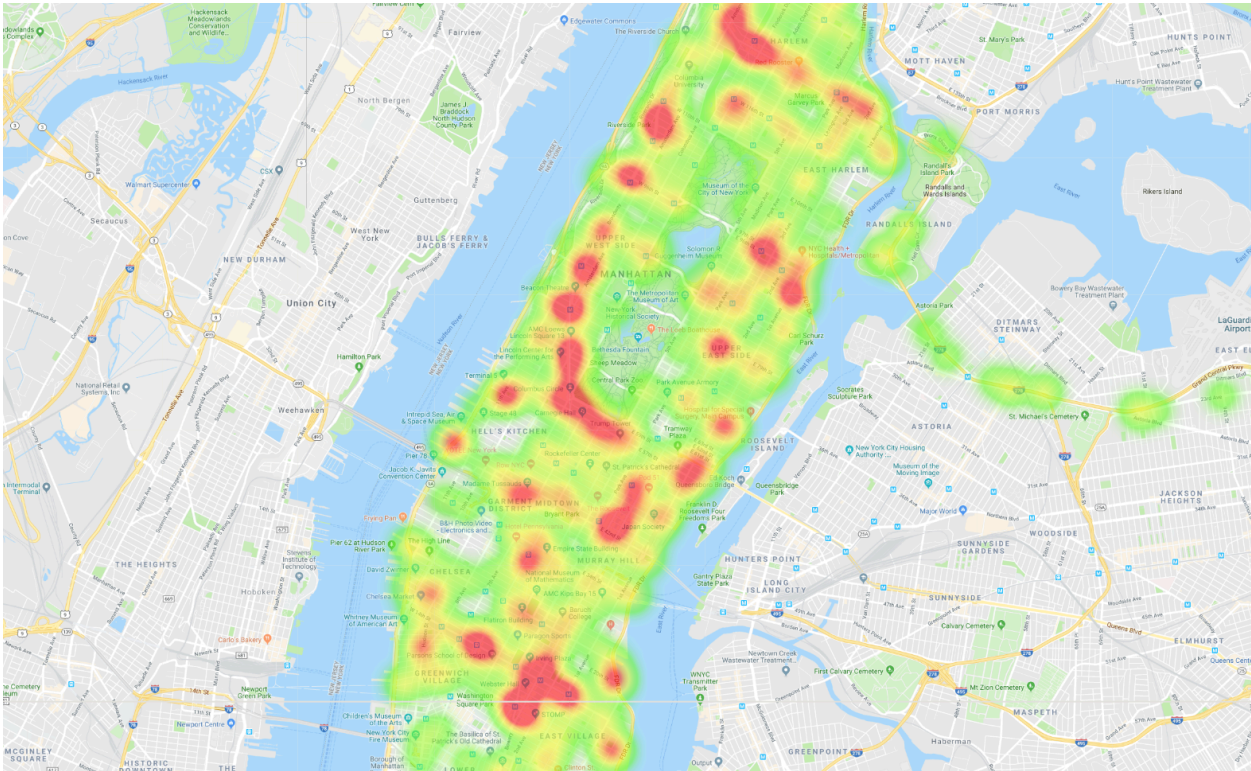
1. 簡介

在本程式碼研究室中，您將瞭解如何搭配 **Java HBase 用戶端**使用 **Cloud Bigtable**。

課程內容

- 避免結構定義設計的常見錯誤
- 匯入序列檔案中的資料
- 查詢資料

完成後，您會看到多張地圖，顯示紐約市的公車資料。舉例來說，您將建立曼哈頓公車行程的熱度圖：



步驟 2 · 約 0 分鐘

2. 關於資料集

您將查看紐約市公車的資料集。當地有超過 300 條公車路線，以及 5,800 輛行駛這些路線的車輛。[我們的資料集](#)是記錄，包含目的地名稱、車輛 ID、緯度、經度、預計抵達時間和排定抵達時間。資料集是由 2017 年 6 月每隔 10 分鐘左右拍攝的快照組成。

3. 結構定義設計

如要讓 Cloud Bigtable 發揮最佳效能，您必須謹慎設計結構定義。Cloud Bigtable 中的資料會自動依字典順序排序，因此只要妥善設計結構定義，查詢相關資料的效率就會非常高。Cloud Bigtable 允許使用點查詢 (依資料列索引鍵) 或資料列範圍掃描進行查詢，並傳回一組連續的資料列。不過，如果結構定義不完善，您可能會發現自己需要拼湊多個資料列查閱作業，更糟的是，您可能需要執行完整的資料表掃描，這類作業非常緩慢。

資料表通常會有數十億列，因此完整掃描資料表可能需要幾分鐘，並佔用其他查詢的資源。

規劃查詢

我們的資料包含各種資訊，但在本程式碼研究室中，您將使用公車的位置和目的地。

有了這些資訊，您就可以執行下列查詢：

- 取得特定時間內單一巴士的位置。
- 取得公車路線或特定公車的當日資料。
- 找出地圖上矩形內的所有公車。
- 取得所有巴士的目前位置 (前提是您即時擷取這項資料)。

這組查詢無法同時以最佳方式完成。舉例來說，如果您依時間排序，就無法在不完整掃描資料表的情況下，根據位置進行掃描。您需要根據最常執行的查詢設定優先順序。

在本程式碼研究室中，您將著重於最佳化及執行下列查詢：

- 取得特定車輛在一小時內的位置資訊。
- 取得一小時內整條公車路線的位置資訊。
- 在一小時內取得曼哈頓所有公車的位置。
- 在一個小時內取得曼哈頓所有公車的最近位置。
- 取得整條公車路線在一個月內的位置。
- 在一小時內取得整條公車路線的所在位置，以及特定目的地的位置。

設計資料列索引鍵

在本程式碼研究室中，您將使用靜態資料集，但會設計可擴充的架構。您將設計結構定義，以便將更多巴士資料串流至資料表，同時維持良好效能。

以下是建議的資料列索引鍵結構定義：

[客運公司/公車路線/時間戳記 (以小時為單位向下取整)/車輛 ID]。每個資料列都有一小時的資料，每個儲存格則包含多個附有時間戳記的資料版本。

在本程式碼研究室中，您會使用一個資料欄系列，以簡化流程。以下是資料的範例檢視畫面。資料會依資料列鍵排序。

資料列索引鍵	cf:VehicleLocation.Latitude	cf:VehicleLocation.Longitude	...
MTA/M86-SBS/1496275200000/NYCT_5824	40.781212 @20:52:54.0040.776163 @20:43:19.0040.778714	-73.961942 @20:52:54.00-73.946949 @20:43:19.00-73.953731 @20:33:46.00	...

	@20:33:46.00		
MTA/M86-SBS/1496275200000/NYCT_5840	40.780664 @20:13:51.0040.788416 @20:03:40.00	-73.958357 @20:13:51.00 -73.976748 @20:03:40.00	...
MTA/M86-SBS/1496275200000/NYCT_5867	40.780281 @20:51:45.0040.779961 @20:43:15.0040.788416 @20:33:44.00	-73.946890 @20:51:45.00-73.959465 @20:43:15.00-73.976748 @20:33:44.00	...
...	...	...	...

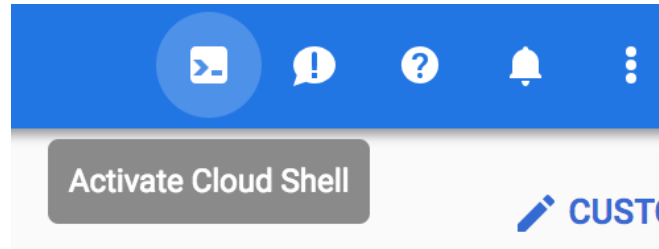
常見錯誤：您可能會認為將**時間**設為資料列鍵中的第一個值是理想做法，因為您可能比較重視近期資料，而且主要會查詢特定時間的資料。不過，[這樣做會導致資料中的熱點](#)，因此您會將「時間」放在第三位，以求折衷。這會增加部分查詢的難度，但您必須這麼做，才能充分發揮 Cloud Bigtable 的效能。此外，您可能不需要一次取得特定時間的所有公車資訊。如要瞭解 Twitter 如何最佳化結構定義，請觀看這場[演講](#)。

步驟 4 · 約 0 分鐘

4. 建立執行個體、資料表和系列

接著，您將建立 Cloud Bigtable 資料表。

首先，請建立新專案。使用內建的 Cloud Shell，按一下右上角的「啟用 Cloud Shell」按鈕即可開啟。



設定下列環境變數，方便複製及貼上 Codelab 指令：

```
INSTANCE_ID="bus-instance"
CLUSTER_ID="bus-cluster"
TABLE_ID="bus-data"
CLUSTER_NUM_NODES=3
CLUSTER_ZONE="us-central1-c"
```

Cloud Shell 已預先安裝本程式碼研究室會用到的工具，包括 [gcloud 指令列工具](#)、[cbt 指令列介面](#)和 [Maven](#)。

執行下列指令來啟用 Cloud Bigtable API。

```
gcloud services enable bigtable.googleapis.com bigtableadmin.googleapis.com
```

執行下列指令來建立執行個體：

```
gcloud bigtable instances create $INSTANCE_ID \
  --cluster=$CLUSTER_ID \
  --cluster-zone=$CLUSTER_ZONE \
  --cluster-num-nodes=$CLUSTER_NUM_NODES \
  --display-name=$INSTANCE_ID
```

完成程式碼實驗室後，請務必刪除執行個體，以免產生週期性費用。

建立執行個體後，請填入 cbt 設定檔，然後執行下列指令，建立資料表和資料欄系列：

```
echo project = $GOOGLE_CLOUD_PROJECT > ~/.cbtrc
echo instance = $INSTANCE_ID >> ~/.cbtrc

cbt createtable $TABLE_ID
cbt createfamily $TABLE_ID cf
```

步驟 5 · 約 0 分鐘

5. 匯入資料

按照下列步驟，從 <gs://cloud-bigtable-public-datasets/bus-data> 匯入本程式碼研究室的一組序列檔案：

執行下列指令來啟用 Cloud Dataflow API。

```
gcloud services enable dataflow.googleapis.com
```

執行下列指令匯入資料表。

```
NUM_WORKERS=$(expr 3 \* $CLUSTER_NUM_NODES)
gcloud beta dataflow jobs run import-bus-data-$(date +%s) \
--gcs-location gs://dataflow-templates/latest/GCS_SequenceFile_to_Cloud_Bigtable \
--num-workers=$NUM_WORKERS --max-workers=$NUM_WORKERS \
--parameters
bigtableProject=$GOOGLE_CLOUD_PROJECT,bigtableInstanceId=$INSTANCE_ID,bigtableTableId
=$TABLE_ID,sourcePattern=gs://cloud-bigtable-public-datasets/bus-data/*
```

監控匯入作業

您可以在 [Cloud Dataflow UI](#) 中監控工作。此外，您也可以透過 Cloud Bigtable 執行個體的[監控使用者介面](#)查看負載。整個匯入程序大約需要 5 分鐘。

步驟 6 · 約 0 分鐘

6. 取得程式碼

```
git clone https://github.com/googlecode-labs/cbt-intro-java.git
cd cbt-intro-java
```

執行下列指令，改用 Java 11：

```
sudo update-java-alternatives -s java-1.11.0-openjdk-amd64 && export
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64/
```

步驟 7 · 約 0 分鐘

7. 執行查詢

您要執行的第一個查詢是簡單的資料列查詢。您會取得 2017 年 6 月 1 日凌晨 12:00 至 1:00 的 M86-SBS 路線公車資料。當時車輛 ID 為 `NYCT_5824`。

有了這些資訊，並瞭解結構定義設計 (巴士公司/巴士路線/時間戳記向下捨入至小時/車輛 ID)，您就能推斷資料列索引鍵為：

```
MTA/M86-SBS/1496275200000/NYCT_5824
```

BusQueries.java

```
private static final byte[] COLUMN_FAMILY_NAME = Bytes.toBytes("cf");
private static final byte[] LAT_COLUMN_NAME = Bytes.toBytes("VehicleLocation.Latitude");
private static final byte[] LONG_COLUMN_NAME = Bytes.toBytes("VehicleLocation.Longitude");

String rowKey = "MTA/M86-SBS/1496275200000/NYCT_5824";
Result getResult =
    table.get(
        new Get(Bytes.toBytes(rowKey))
            .addColumn(COLUMN_FAMILY_NAME, LAT_COLUMN_NAME)
            .addColumn(COLUMN_FAMILY_NAME, LONG_COLUMN_NAME));
```

結果應包含該小時內公車的最新位置。但您想查看所有位置，因此請在 GET 要求中設定版本數量上限。

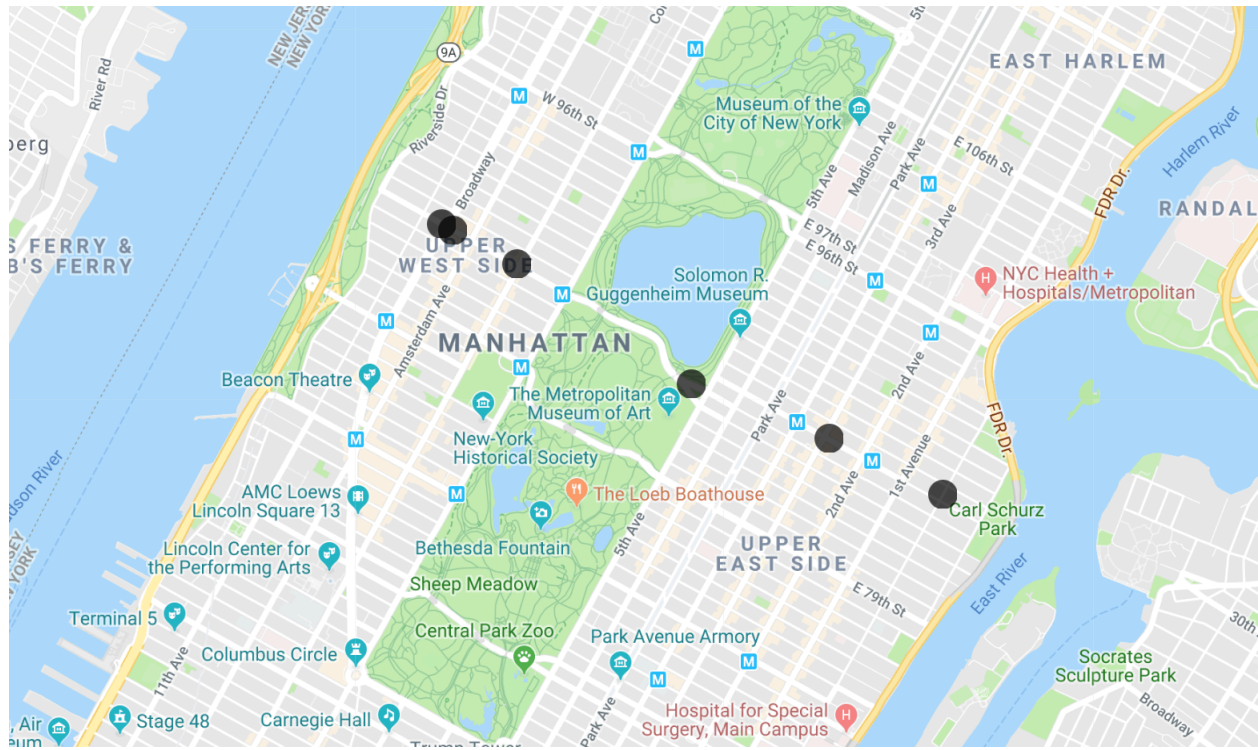
BusQueries.java

```
Result getResult =
    table.get(
        new Get(Bytes.toBytes(rowKey))
            .setMaxVersions(Integer.MAX_VALUE)
            .addColumn(COLUMN_FAMILY_NAME, LAT_COLUMN_NAME)
            .addColumn(COLUMN_FAMILY_NAME, LONG_COLUMN_NAME));
```

在 Cloud Shell 中執行下列指令，取得該公車在一小時內的緯度和經度清單：

```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \
-Dquery=lookupVehicleInGivenHour
```

您可以將緯度和經度複製並貼到 [MapMaker 應用程式](#)，以視覺化呈現結果。幾層之後，系統會要求你建立免費帳戶。你可以建立帳戶，也可以直接刪除現有圖層。本程式碼研究室的每個步驟都附有視覺化內容，方便您跟著操作。以下是第一次查詢的結果：



步驟 8 · 約 0 分鐘

8. 執行掃描

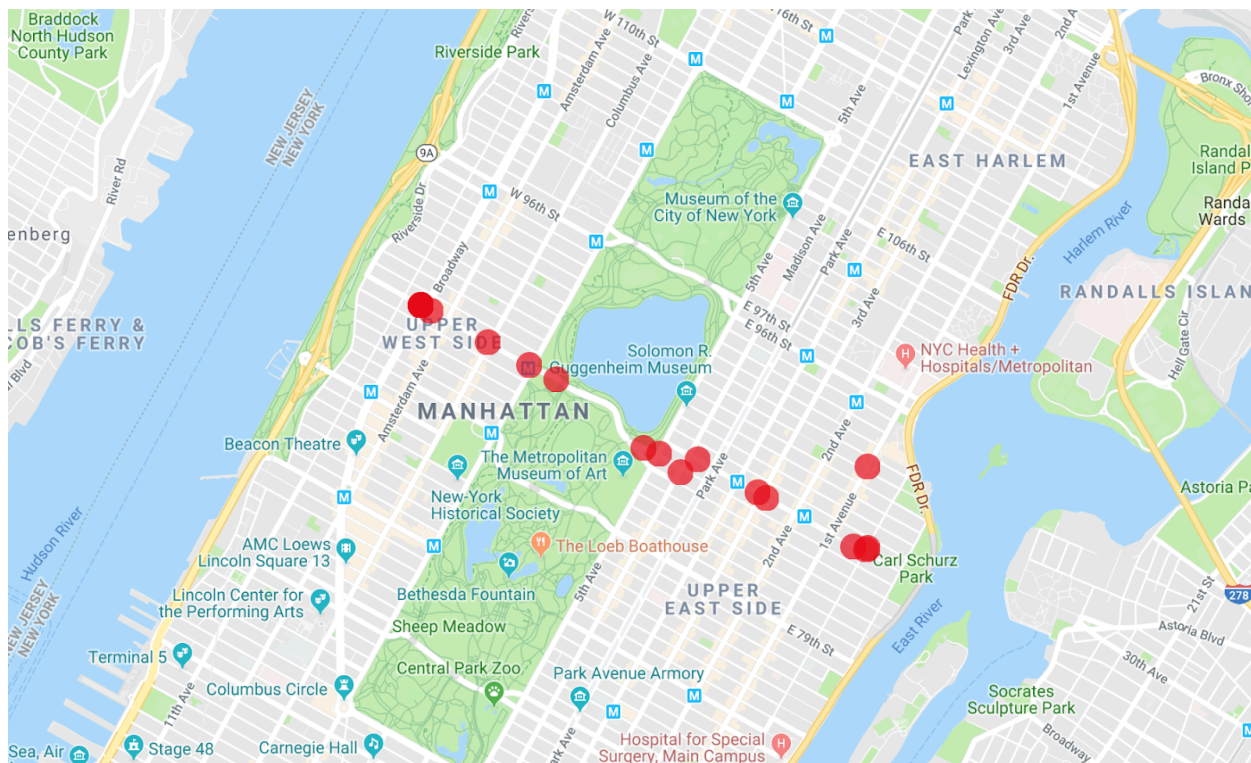
現在，讓我們查看該小時的公車路線所有資料。掃描碼與取得代碼非常相似。您為掃描器提供起始位置，然後指出您只需要時間戳記 1496275200000 所標示的小時內，M86-SBS 公車路線的資料列。

BusQueries.java

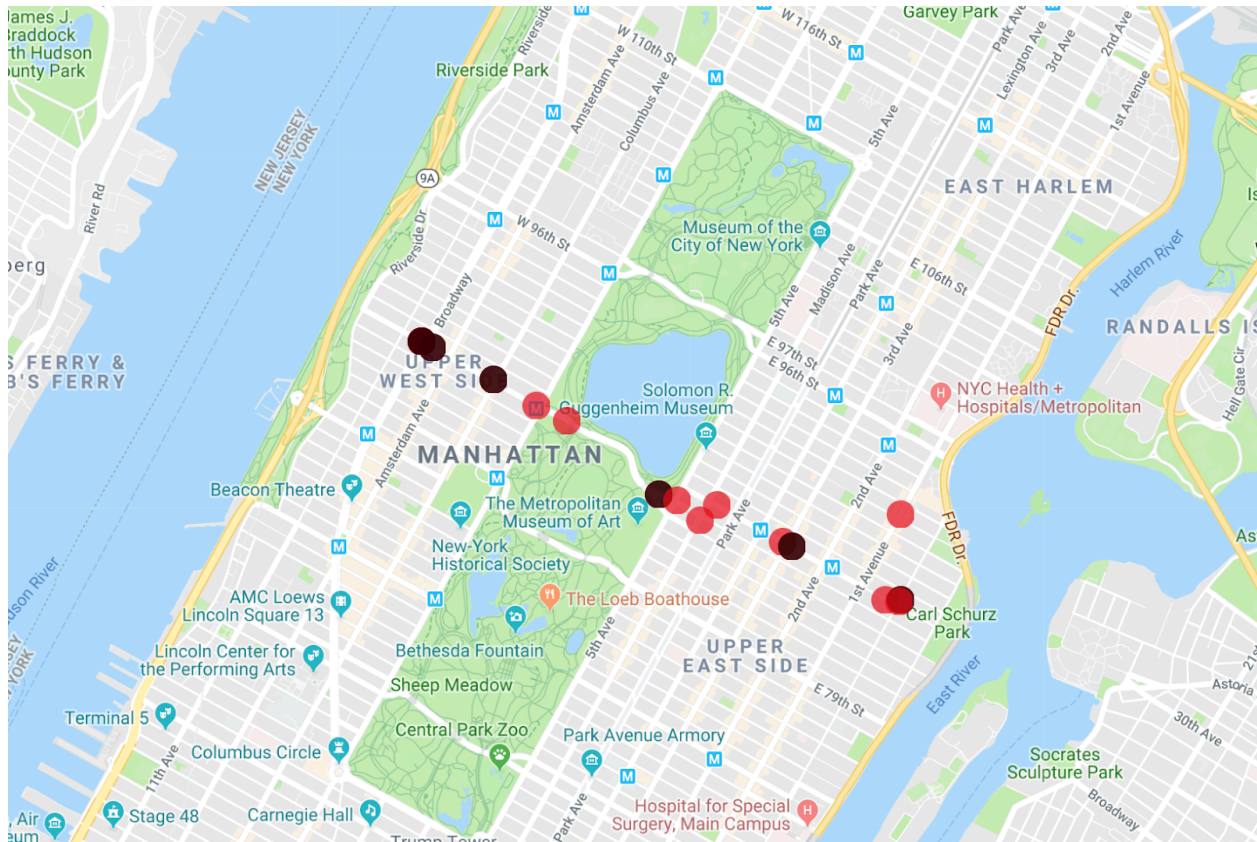
```
Scan scan;  
scan = new Scan();  
scan.setMaxVersions(Integer.MAX_VALUE)  
    .addColumn(COLUMN_FAMILY_NAME, LAT_COLUMN_NAME)  
    .addColumn(COLUMN_FAMILY_NAME, LONG_COLUMN_NAME)  
    .withStartRow(Bytes.toBytes("MTA/M86-SBS/1496275200000"))  
    .setRowPrefixFilter(Bytes.toBytes("MTA/M86-SBS/1496275200000"));  
ResultScanner scanner = table.getScanner(scan);
```

執行下列指令即可取得結果。

```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \  
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \  
-Dquery=scanBusLineInGivenHour
```



地圖製作工具應用程式可同時顯示多個清單，因此您可以查看哪些巴士是您第一次查詢時的車輛。



這項查詢的有趣修改是查看 M86-SBS 巴士路線整個月的資料，這非常容易做到。從開始列和前置字元篩選器中移除時間戳記，即可取得結果。

BusQueries.java

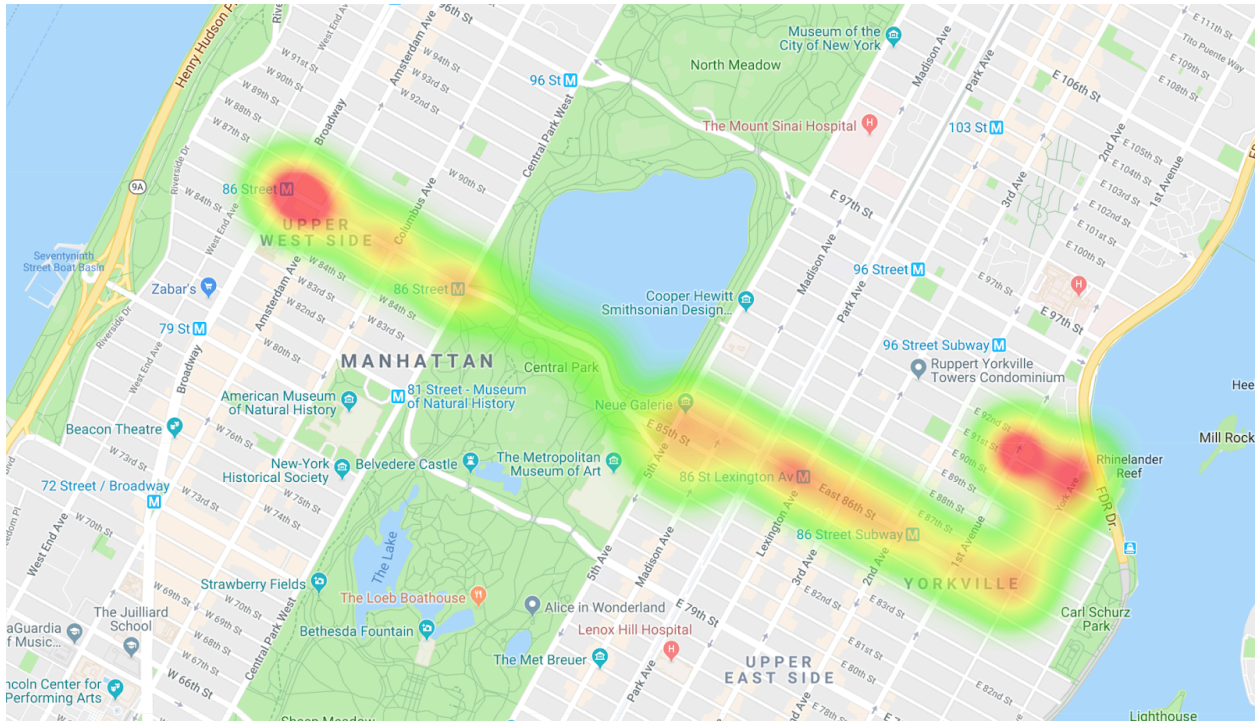
```
scan.withStartRow(Bytes.toBytes("MTA/M86-SBS/"))
    .setRowPrefixFilter(Bytes.toBytes("MTA/M86-SBS/"));

// Optionally, reduce the results to receive one version per column
// since there are so many data points.
scan.setMaxVersions(1);
```

執行下列指令即可取得結果。(結果清單會很長)。

```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \
-Dquery=scanEntireBusLine
```

如果將結果複製到 MapMaker，即可查看公車路線的熱視圖。橘色圓點代表停靠站，亮紅色圓點則代表路線的起點和終點。



9. 介紹篩選器

接著，您將篩選出往東和往西的公車，並分別建立熱度圖。

注意：這個篩選器只會檢查最新版本，因此請將最大版本數設為 1，確保結果符合篩選條件。進一步瞭解如何[設定篩選器](#)。

BusQueries.java

```
Scan scan;
ResultScanner scanner;
scan = new Scan();
SingleColumnValueFilter valueFilter =
    new SingleColumnValueFilter(
        COLUMN_FAMILY_NAME,
        Bytes.toBytes("DestinationName"),
        CompareOp.EQUAL,
        Bytes.toBytes("Select Bus Service Yorkville East End AV"));

scan.setMaxVersions(1)
    .addColumn(COLUMN_FAMILY_NAME, LAT_COLUMN_NAME)
    .addColumn(COLUMN_FAMILY_NAME, LONG_COLUMN_NAME);
scan.withStartRow(Bytes.toBytes("MTA/M86-SBS/"))
    .setRowPrefixFilter(Bytes.toBytes("MTA/M86-SBS/"));
scan.setFilter(valueFilter);
scanner = table.getScanner(scan);
```

執行下列指令，取得往東的公車結果。

```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \
-Dquery=filterBusesGoingEast
```

如要取得往西的公車，請變更 valueFilter 中的字串：

BusQueries.java

```
SingleColumnValueFilter valueFilter =
    new SingleColumnValueFilter(
        COLUMN_FAMILY_NAME,
        Bytes.toBytes("DestinationName"),
        CompareOp.EQUAL,
        Bytes.toBytes("Select Bus Service Westside West End AV"));
```

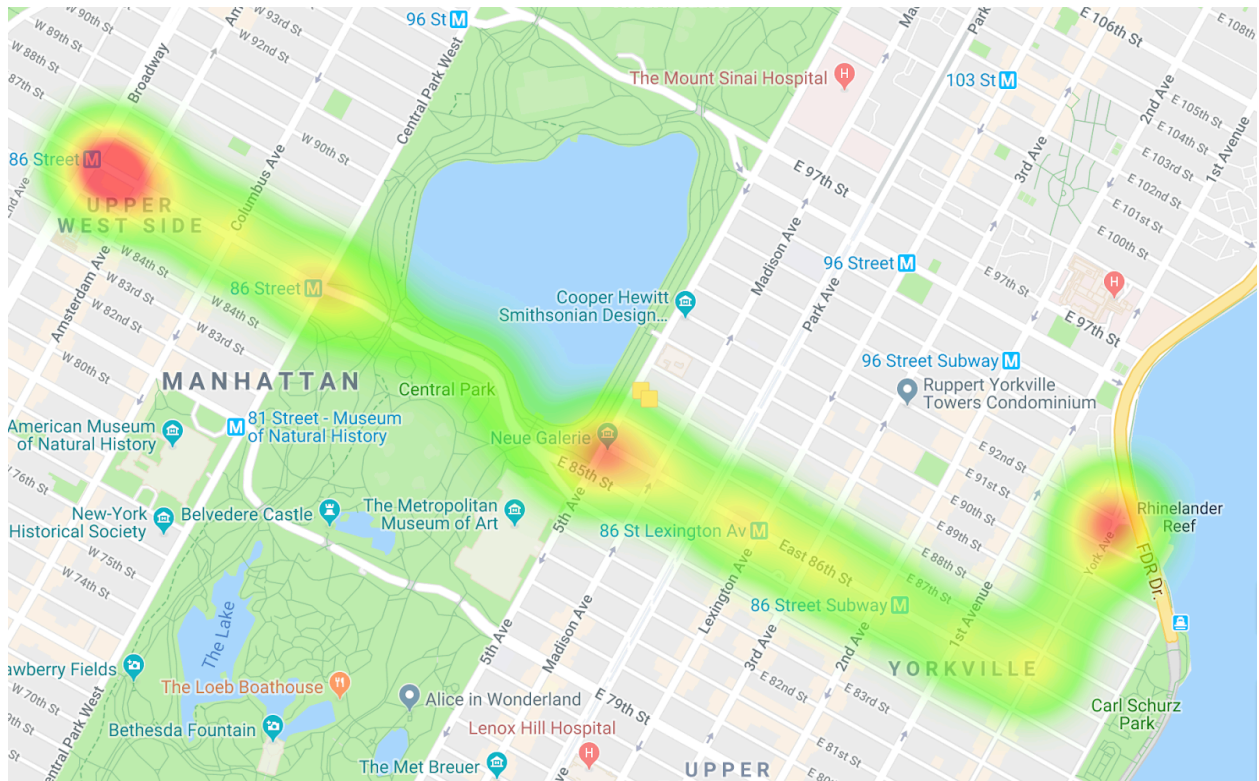
執行下列指令，取得往西的公車結果。


```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \
-Dquery=filterBusesGoingWest
```

往東的公車



往西的公車



比較這兩張熱感應圖，即可看出路線差異，以及配速差異。其中一種解讀方式是，在往西的路線上，公車停靠次數較多，尤其是在進入中央公園時。而往東的巴士則沒有太多瓶頸。

步驟 10 · 約 0 分鐘

10. 執行多範圍掃描

最後一個查詢會處理您關心某個區域內多條公車路線的情況：

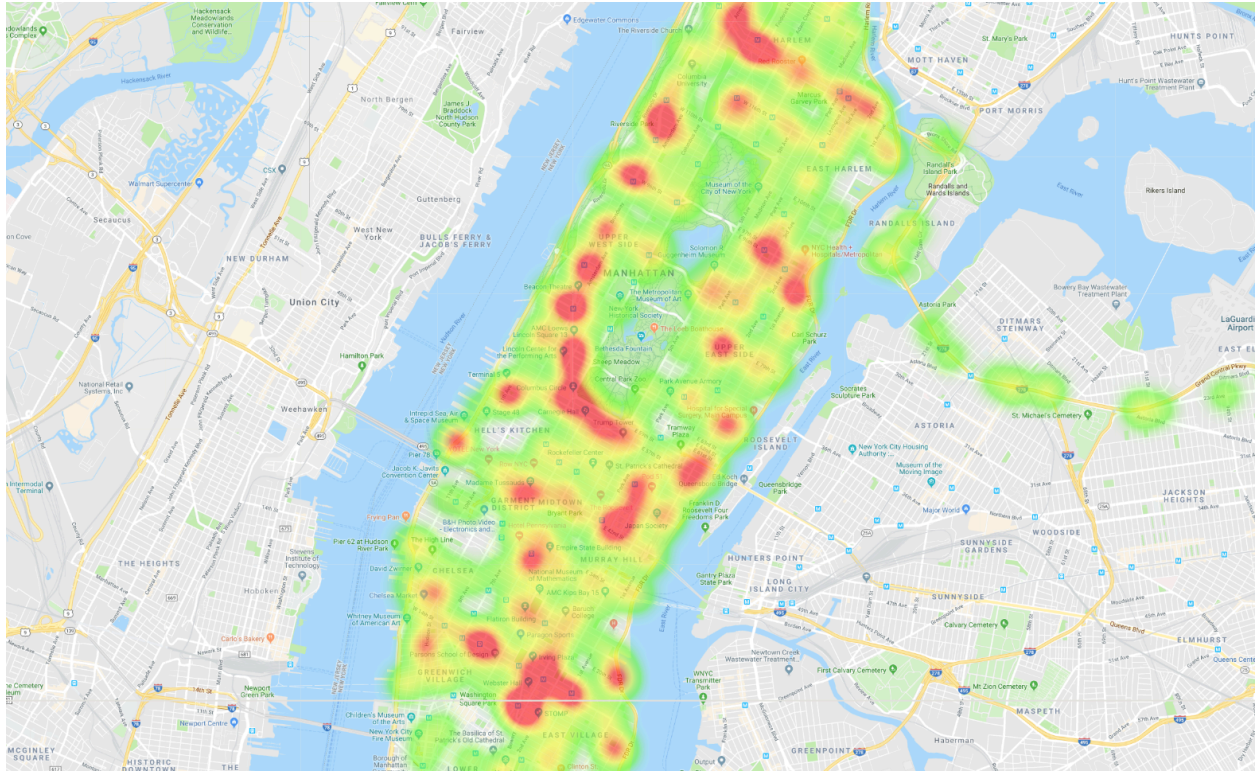
BusQueries.java

```
private static final String[] MANHATTAN_BUS_LINES = {"M1", "M2", "M3", ...

Scan scan;
ResultScanner scanner;
List<RowRange> ranges = new ArrayList<>();
for (String busLine : MANHATTAN_BUS_LINES) {
    ranges.add(
        new RowRange(
            Bytes.toBytes("MTA/" + busLine + "/1496275200000"), true,
            Bytes.toBytes("MTA/" + busLine + "/1496275200001"), false));
}
Filter filter = new MultiRowRangeFilter(ranges);
scan = new Scan();
scan.setFilter(filter);
scan.setMaxVersions(Integer.MAX_VALUE)
    .addColumn(COLUMN_FAMILY_NAME, LAT_COLUMN_NAME)
    .addColumn(COLUMN_FAMILY_NAME, LONG_COLUMN_NAME);
scan.withStartRow(Bytes.toBytes("MTA/M")).setRowPrefixFilter(Bytes.toBytes("MTA/M"));
scanner = table.getScanner(scan);
```

執行下列指令即可取得結果。

```
mvn package exec:java -Dbigtable.projectID=$GOOGLE_CLOUD_PROJECT \
-Dbigtable.instanceID=$INSTANCE_ID -Dbigtable.table=$TABLE_ID \
-Dquery=scanManhattanBusesInGivenHour
```



11. 完成設定

清除所用資源，以免產生費用

如要避免系統向您的 Google Cloud Platform 帳戶收取您在本程式碼研究室中所用資源的相關費用，請刪除執行個體。

```
gcloud bigtable instances delete $INSTANCE_ID
```

涵蓋內容

- 結構定義設計
- 設定執行個體、資料表和系列
- 使用 Dataflow 匯入序列檔案
- 使用查詢、掃描、掃描並套用篩選器，以及多範圍掃描功能查詢

後續步驟

- 如要進一步瞭解 Cloud Bigtable，請參閱[說明文件](#)。
 - 自行試用其他 Google Cloud Platform 功能。並參考我們的[教學課程](#)。
 - 瞭解如何[使用 OpenTSDB 整合功能監控時間序列資料](#)
-